

Consegna 4 - Setup del Cluster HPC

Lab 1 - Caratterizzazione Hardware dell'Ambiente di Calcolo

L'obiettivo è di determinare le specifiche tecniche e le prestazioni teoriche di picco dei nodi di calcolo messi a disposizione sul cluster HPC di ateneo. In particolare, l'analisi si concentra su un nodo della partizioni `cpu_guest` e sulla GPU NVIDIA Tesla P100.

Analisi del Nodo CPU (`cpu_guest`)

Per estrarre le caratteristiche del nodo di calcolo della partizione CPU, è stato sottomesso il job batch `1sCPU.slurm`. Il sistema slurm ha allocato il nodo di calcolo **wn99**.

Dall'output del comando `1sCPU` si deducono le seguenti specifiche hardware:

- **Modello CPU:** AMD EPYC 7282 16-Core Processor.
- **Architettura:** x86_64 (supporta modalità a 32-bit e 64-bit).
- **Layout Core/Socket:** il nodo presenta 2 Socket, ciascuno equipaggiato con 16 Core, per un totale di 32 Core fisici online (0-31). Il threading per core è impostato a 1 (Hyper-Threading disattivato).
- **Frequenza di Clock:** la frequenza minima è di 1500.00 MHz, mentre la frequenza massima di clock (Max Turbo) raggiunge i **2800.00 MHz (2.8 GHz)**.
- **Gerarchia della Cache:**
 - L1 Data Cache / Instruction Cache: 32 KB per core.
 - L2 Cache: 512 KB per core.
 - L3 Cache: 16384 KB (16 MB) condivisa.
- **Architettura di Memoria (NUMA):** Sono presenti 2 nodi NUMA. Il `node0` gestisce i core 0-15, mentre il `node1` gestisce i core 16-31.

Calcolo delle Prestazioni di Picco Teoriche

Le prestazioni di picco teoriche di un core della CPU in virgola mobile si calcolano con:

$$\text{Flops} = \text{Frequenza (GHz)} \times \text{Instruction/Cycle (IPC)}$$

Analizzando i flag della CPU, è presente il supporto alle istruzioni AVX2 (Advanced Vector Extensions 2). L'architettura AMD Zen 2 esegue i vettori AVX2 a 256 bit effettuando 2 FMA (Fused Multiply-Add) per ciclo di clock. Poiché un'operazione FMA conta come 2 operazioni (una moltiplicazione e una somma) su vettori da 8 elementi FP32 (256 bit / 32 bit):

$$\text{IPC (FP32)} = 2 \text{ FMA} \times 8 \text{ elementi} = 16 \text{ FLOPS/ciclo per core}$$

Utilizzando la frequenza massima di 2.8 GHz:

$$\text{Prestazioni di picco per singolo core} = 2.8 \text{ GHz} \times 16 = 44.8 \text{ GFLOPS}$$

Moltiplicando per la totalità dei core del nodo (32 core attivi):

Prestazioni di picco dell'intero nodo = $44.8 \text{ GFLOPS} \times 32 = 1.433 \text{ TFLOPS}$

Analisi del Nodo GPU

È stato eseguito l'applicativo `deviceQuery` sottomesso tramite slurm sul nodo **wn41**. Il sistema ha rilevato la seguente scheda grafica dedicata:

- **Modello GPU:** NVIDIA Tesla P100-PCIE-12GB.
- **Architettura CUDA:** Compute Capability 6.0 (Architettura Pascal).
- **Risorse di Calcolo:**
 - Presenza di **56 Multiprocessori (SM)**.
 - Ciascun SM contiene 64 CUDA Cores , per un totale complessivo di **3584 CUDA Cores**.
- **Frequenze e Memoria:**
 - Frequenza di clock massima (GPU Max Clock rate): **1329 MHz (1.33 GHz)**.
 - Memoria Globale Totale: 12187 MBytes (~12 GB) ad alta larghezza di banda con un bus di memoria a **3072-bit**.
 - L2 Cache Size: 3,145,728 byte (~3 MB).
- **Limiti e Proprietà dei Thread:**
 - Dimensione del Warp: 32 thread.
 - Massimo numero di thread per blocco: 1024.
 - Massimo numero di thread per multiprocessore (SM): 2048.
 - Memoria condivisa (Shared Memory) per blocco: 49152 byte (48 KB).

Calcolo delle Prestazioni di Picco Teoriche (GPU)

La performance teorica in precisione singola (FP32) della GPU si calcola moltiplicando il numero complessivo di CUDA Cores per la frequenza massima di clock, considerando che l'architettura esegue un'istruzione FMA (2 operazioni) per ciclo:

$$\text{Peak FP32} = \text{Numero CUDA Cores} \times \text{Frequenza di Clock (GHz)} \times 2$$

Sostituendo i dati reali ottenuti:

$$\text{Peak FP32} = 3584 \times 1.329 \text{ GHz} \times 2 = 9526.27 \text{ GFLOPS} \approx 9.53 \text{ TFLOPS}$$

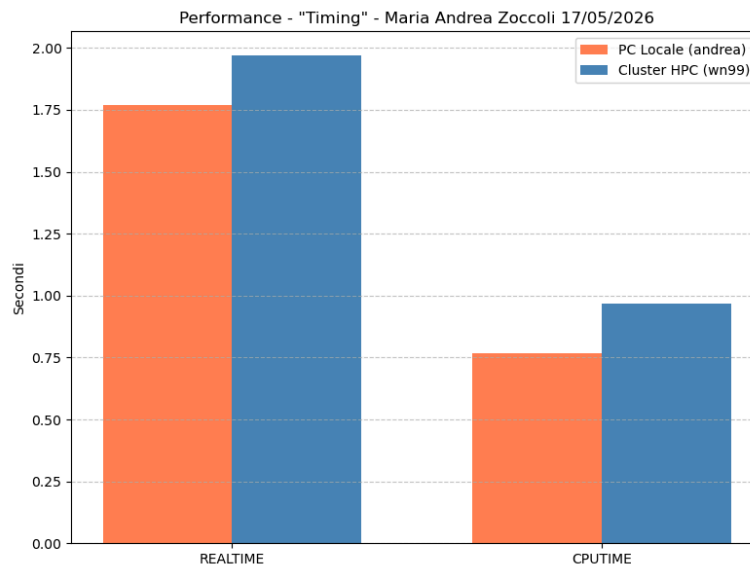
Lab 2 - Valutazione delle Performance Hardware

Il secondo laboratorio ha lo scopo di misurare le prestazioni effettive dei sistemi di calcolo analizzati nel lab 1, testandone le capacità di elaborazione (CPU e GPU) e le prestazioni di comunicazione (network).

Profilazione dei Tempi di Esecuzione (Timing)

Prima di procedere con i benchmark computazionali, è stato eseguito un test preliminare per valutare le funzioni di misurazione del tempo di sistema (utilizzando le chiamate POSIX `clock_gettime()`). Lo scopo è isolare il tempo reale di esecuzione dal tempo effettivo dedicato dalla CPU al processo.

- **REALTIME:** misura il tempo "wall-clock" totale trascorso dall'avvio alla fine dell'esecuzione, includendo l'overhead del sistema operativo, i tempi di attesa I/O e lo scheduling.
- **CPUTIME:** misura esclusivamente il tempo che la CPU ha impiegato attivamente per elaborare le istruzioni del programma, escludendo i tempi di inattività o di background.



Come si osserva nel grafico, in entrambi gli ambienti di calcolo il valore di REALTIME risulta coerentemente maggiore del CPUTIME. Questo comportamento è tipico dei sistemi operativi multitasking, dove il processo non ha il controllo esclusivo e ininterrotto dell'hardware. Si nota inoltre una leggera variazione dei tempi assoluti tra il PC locale e il nodo di calcolo, dovuta alle differenze di frequenza operativa dei processori e al carico di background gestito dallo scheduler sul cluster condiviso.

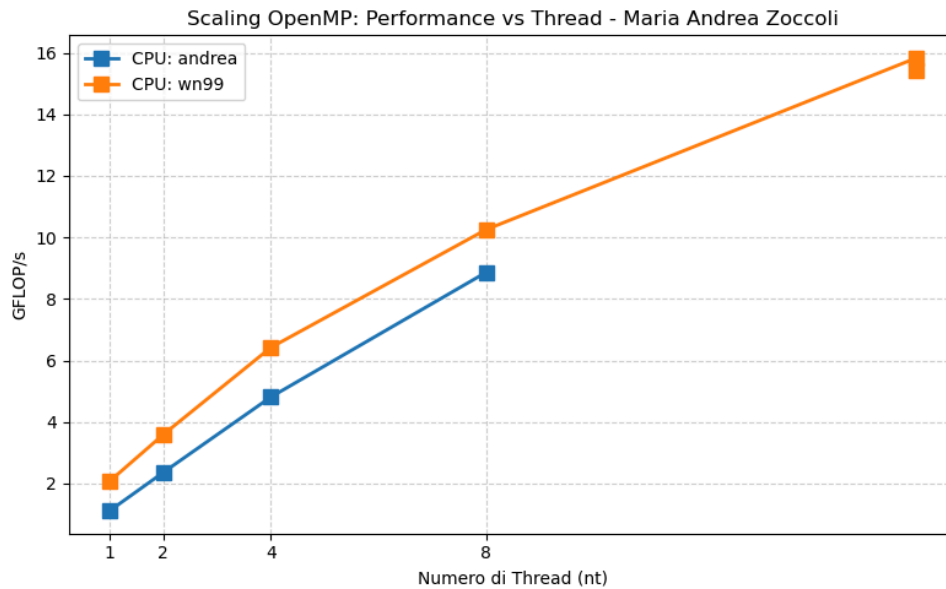
Prestazioni di Calcolo (Matrix Multiplication)

Per valutare le performance di calcolo è stato utilizzato un algoritmo di moltiplicazione tra matrici quadrate dense di dimensione ($N = 1024$). La metrica di riferimento è il numero di operazioni in virgola mobile al secondo (GFLOP/s), ricavato dalla formula $FLOP/s = 2N^3/T$, dove T è il tempo di esecuzione misurato.

Scalabilità su CPU (OpenMP)

Il primo test ha valutato la scalabilità dell'algoritmo Naive parallelizzato tramite la libreria OpenMP, incrementando il numero di thread assegnati al calcolo (1, 2, 4, 8) sia sul PC locale sia sul nodo del cluster. Come si evince dal grafico Scaling OpenMP, l'andamento delle prestazioni mostra un incremento coerente con i paradigmi del calcolo parallelo in memoria condivisa:

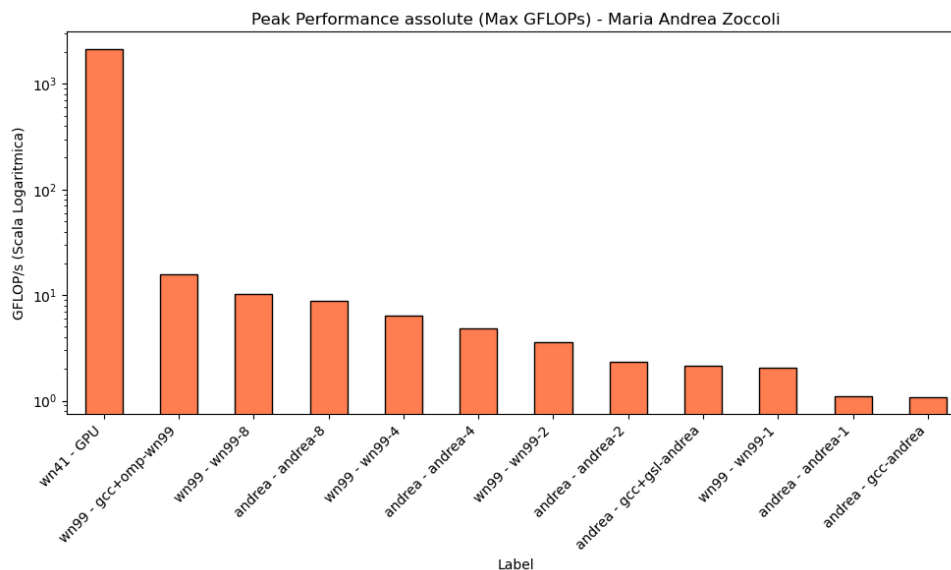
- **PC locale:** mostra un incremento lineare delle prestazioni fino a 4 thread, beneficiando della presenza dei 4 core fisici della CPU. Nel passaggio a 8 thread (core logici tramite Hyper-Threading), la curva tende ad appiattirsi a causa dei limiti strutturali della condivisione delle risorse fisiche sottostanti.
- **Cluster HPC:** avendo a disposizione un nodo con 32 core fisici reali, la CPU gestisce lo scaling sui primi 8 thread in modo nativo e senza colli di bottiglia architetturali, mantenendo un gradiente di crescita più stabile e performance assoluto superiori.



Prestazioni di Picco Assolute (CPU vs GPU)

Il secondo test mette a confronto l'efficienza delle ottimizzazioni software su CPU (algoritmo Naive vs libreria scientifica ottimizzata GSL `gsl_blas_sgemm()`) con la potenza di calcolo parallela massiccia offerta dalla GPU del cluster.

- **Ottimizzazioni CPU:** sia in ambiente locale che sul cluster, l'adozione di librerie matematiche dedicate (GSL) o di costruttori multithread (OpenMP a 8 thread) garantisce un salto prestazionale netto rispetto all'approccio sequenziale Naive.
- **Accelerazione GPU:** l'introduzione nel confronto della GPU NVIDIA Tesla P100 evidenzia un divario prestazionale di circa due ordini di grandezza, visibile chiaramente sulla scala logaritmica. Sfruttando l'architettura SIMT tramite le API CUDA, la GPU elabora la matrice parallelamente sui suoi 3584 core, superando ampiamente la soglia dei 1000 GFLOP/s.



Prestazioni di Rete

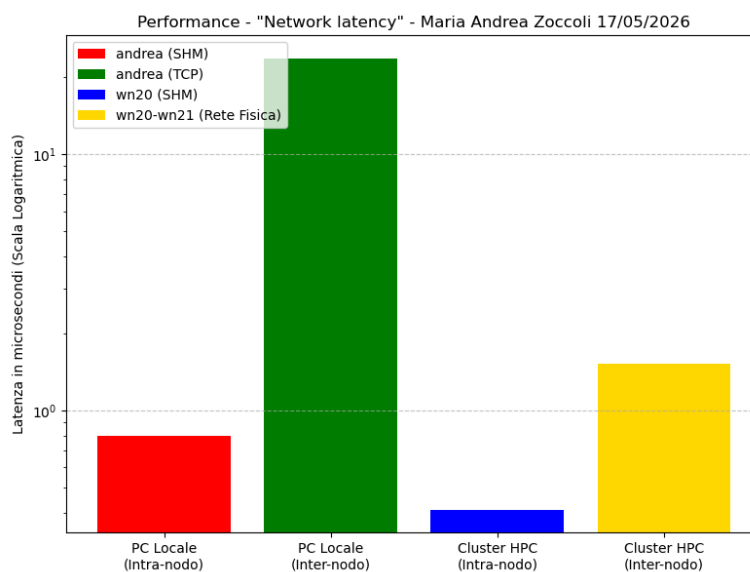
La caratterizzazione del sottosistema di rete valuta l'efficienza della comunicazione Message Passing (MPI) tra due processi distribuiti. Il tempo totale di trasferimento di un messaggio di N byte è governato dalla relazione lineare

$T = L + N/B$, dove L rappresenta la latenza (tempo di overhead fisso per l'avvio del messaggio) e B la larghezza di banda passante.

Latenza di Rete

Il benchmark del tempo di round-trip per messaggi a payload minimo evidenzia l'impatto dei canali fisici e logici:

- **Canale intra-nodo (Shared Memory- SHM):** quando i due processi MPI cooperano sullo stesso nodo (sia sul PC locale sia sul nodo del cluster), i dati non transitano su una rete fisica ma vengono mappati in RAM dal sistema operativo. La latenza è minima e si attesta su frazioni di microsecondo.
- **Canale Inter-nodo (rete fisica vs emulazione):** il cluster HPC si avvale di interconnessioni hardware ad altissima velocità, mantenendo la latenza inter-nodo molto bassa ($\sim 1.5 \mu s$). Al contrario, il PC locale registra un valore critico ($20 \mu s$): l'assenza di una vera rete inter-nodo costringe OpenMPI a emulare la comunicazione tramite stack TCP/IP su interfaccia di loopback locale, accumulando un pesante overhead software di sistema.



Larghezza di Banda

Il test di bandwidth descrive la capacità della rete di saturare il canale all'aumentare della dimensione del messaggio (in byte).

- Per messaggi piccoli, la velocità (MBps) è fortemente limitata dai tempi di latenza descritti in precedenza.
- Al crescere del payload, l'efficienza della trasmissione aumenta. In configurazione a memoria condivisa (SHM), le curve stimate rispecchiano le ottime velocità di trasferimento interne dei rispettivi bus di memoria RAM.
- Nella comunicazione esterna, l'infrastruttura di rete del cluster mostra una curva di crescita stabile e continua che satura verso i 4000 MBps, garantendo la larghezza di banda necessaria per evitare che i nodi di calcolo rimangano in stato di idle durante lo scambio di grandi porzioni di dati distribuiti. La controparte TCP emulata su PC locale si rivela inefficiente, instabile e fortemente frastagliata.

Performance - "Network bandwidth" - Maria Andrea Zoccoli 17/05/2026

